Course 7 - PaaS & Managed Services

PaaS

- Platform as a Service
- noi tinem cont doar de Application and Data
- bazat pe IaaS
- servicii pentru challenges common
- managed services
- wide range of services: - compute - App Engine, Cloud Functions - storage - aws s3, google cloud storage - monitoring - aws cloudwatch - user management - aws cognito - Encryption - aws kms - combinations of services - aws elastic beanstalk PROS: - focus on business than features - dedicated team for maintenance - different billing - can be cheaper CONS: - less control - more costly to run - vender lock in

When to use

```
- fast prototyping
- small team
- u understand the lock in
```

When not to use

```
- exotic products: banking
- ultra high performance - latency sensitive
- control security is must - most PaaS provide better security, but we cannot be
sure
- when working with special licenses - PaaS may not support
```

Examples: - Gooooooogle App Engine - AWS Elastic Beanstalk - Azure App Service

Envirorment Parity

```
- unul din cele 12 factori
- local dev stage prod sa fie similire
```

Containers

- unit containing the app and all its dependencies Useful for: - create identical/similar envs - quickly provisioning dependencies locally

Docker

- containerization platform
- Namespaces : looks more like a complete VM, isolated from the host
- Cgroups: control the resources that a process can use
- Union file systems: layers of file systems Images: similar to classes that make heavy use of inheritance
- stored locally on container registries Containers: - running based on images - can be stopped, started, deleted - we can execute commands Services: - manage multiple containers - can be scaled - load balancing Volumes: - needed since containers are ephemeral (they can be deleted) and isolated - persist data separate from the container Networks: - containers can communicate with each other - control network isolation

Kubernetes

- orchestration tool for containers
- useful for :
    - running containers in a cluster
    - optimal resource usage
    - large variable workloads
    - roling updates

Cluster

- underlying infrastructure (any IaaS and on Premises)

Nodes

```
- VM s available to kubernets
```

Pods

```
- smallest unit of kubernetes
- runs one or more related containers
- multiple pods can run on a node
- managed by other components usual
```

Services

```
- pods cannot be reached by default
- services expose pods to the outside world
- multiple types: ClusterIP,NodePort, LoadBalancer, ExternalName
```

PersistenVolumes

```
- physical storage in the cluster
- consumed by pods through PersistentVolumeClaims
```

Course 8 - Databases

Types of Workloads:

1. Online Transactional Processing (OLTP):

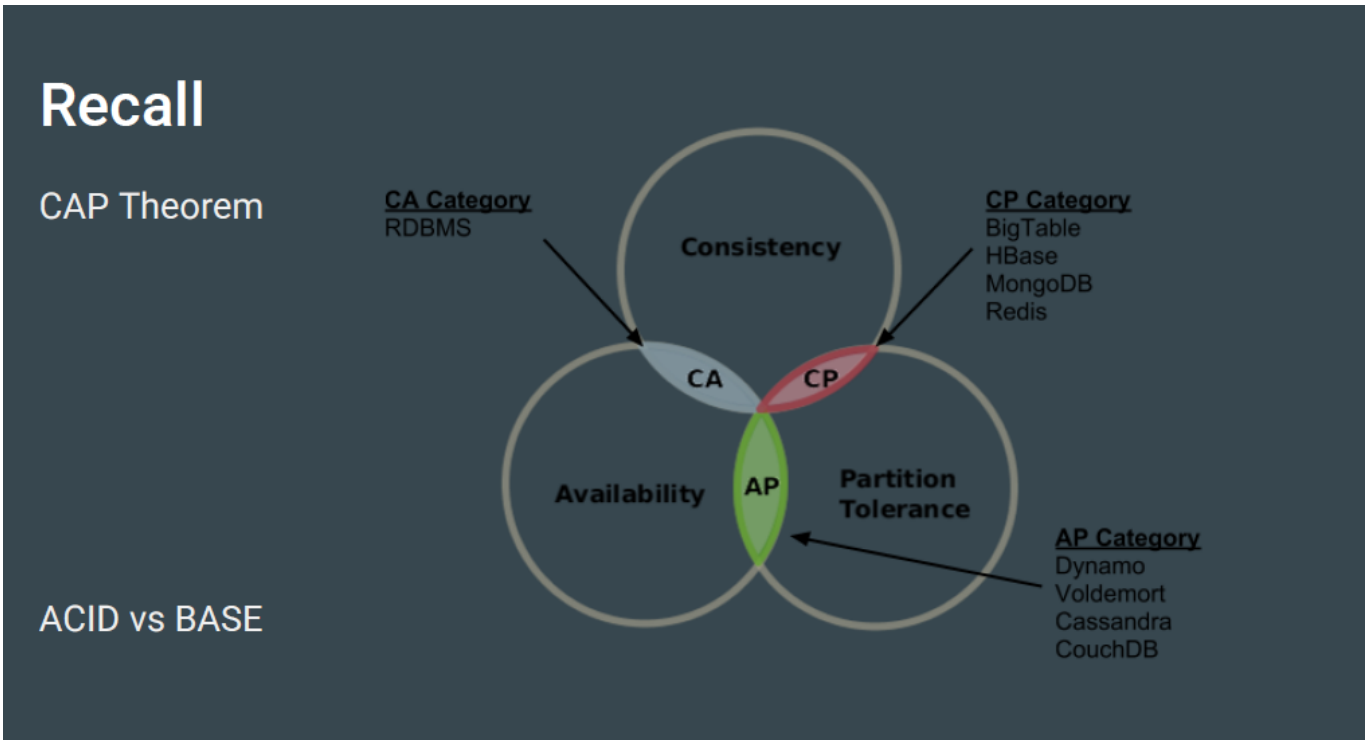
- banking, ERP, booking billing Caracteristici: - transactions -> doar unul reuseste - known access patterns - write heavy - known data scehma - operatii scurte - back up

2. Online Analytical Processing (OLAP):

- inainte era relevant doar pentru large enterprises
- based on data warehouse
- analyze sales in a certain period (eg) Caracteristici: - access patterns/user queries not known entirely - read heavy - large data volumes - possibly long ops - data consistency & back up nu e crucial

3. Decision Support System (DSS):
    - similar cu OLAP
    - queries might be known tho since the beginngin
    - data spans longer timeframes

Types of Databases:

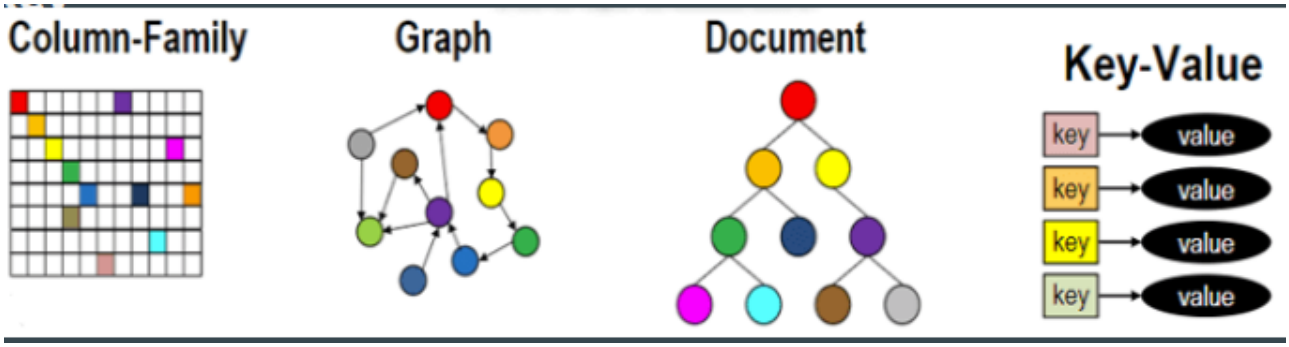


Relational

- SQL- Structured Query Language RDBMS - Relational Database Management System
- optimizat pentru storage (reduced data dup- normalizare)

NoSQL

- wide range of db types
- optimized for compute and scalability



Key-Value

- data e accesata bazata pe o keye
- poate fi scalat infint
- ultra low and consistent latency
- high and consistent throughput
- Redis, DynamoDB

Column

Column stores

- salveaza tabelele in coloane nu in randuri
- serializeaza valoriile unei coloane impreuna similar cu indexul din row oriented db
- reduces amount of data read by compressing similar data pe coloane si citind doar datele necesare eg: Cassandra, HBase, MariaDB ColumnStore

Wide Column

- arhitetura uses persistent sparse matrix (row key, column key, timestamp, value) multi dimensional map in format tabular meant for massive scalability
- eg: Google BigTable, Apache HBase

Document

- stores data in documents
- data e o colectie de documente contintant perechi key value
- valoriile pot fi si scalare dar si nested docs sau liste
- attr names are dinamically defined
- eg: MongoDB, Couchbase, CouchDB,Elasitcsearch

Graph

- topographical data models pentru date
- data points sunt noduri si conexiunile dintre ele sunt edges
- noduriile pot avea propietati
- relatiile sunt numite si directed edges si avem start si end
- relatiile pot avea propietati
- eg: Neo4j, Amazon Neptune, OrientDB

Time-Series

- time stamped data
- data care vine e aproape mereu salvata ca entry nou
- ajunge in time order time series are generally append only data could be measurments or event tracked

Search

- search data content
- index categorizand caract asem dintre date si faciliteaza capabilitatea
- optimizat pentru date lungi , semistructured sau ne strturate
- full text seach complex search ranking of search eg: ElasticSearch,Solr

Ledger

- immutable : nu poti schimba trecutul . totul se face cu append
- transparent : poti vedea trecutul si poti lua log uriile
- verifiable - validare schimbari istorice

Securitate

- admin user or auth nu e default
- weak password
- client comunica cu serverul via plaintext(Mongo)
- nu putem folosi encryption externe ca si ldap
- lack of encrypotuon support for the data files
- weak authentication betwee client and servers
- sql linjection
- dos attack

Alegere DB

- purpose?
- acces patterns?
- performance
- volume
- fiecare are db ul lui?
- scale intended reac?
- one model fit all?

Migrare CLOUD

LIFT & SHIFT

AWS DB Migration SErvice support homogeneous si heterogeneource source.targe db drs pot fi fol

Course 9 - Securitate

CIA triad

Confidentiality

- doar entitatiile autorizate au access la data

Integrity

- doar entitatiile autorizate pot modifica data

Availability

- nu are sens sa ai sistem well-guarded daca nu e disponibil
- cutremurele pot afecta data centerul

Standarde:

CSA (Cloud Security Alliance) - Best practices for cloud security ● ISO 27001 - Information Security Management ● ISO 27017 - 27001 in the context of cloud ● ISO 27018 - Personal data protection in cloud ● PCI DSS - Card payments security ● HIPAA - Health information protection

Shared Responsibility Model

- AWS: AWS is responsible for the security of the cloud. You are responsible for security in the cloud.
- Google: Google is responsible for the security of the cloud. You are responsible for the security of your data in the cloud.

Infrastructure security

- as opposed to application security Concerned with:
    - network arhitecture
    - cloud users
    - app/system perm
    - resource access permissions e o paradigma diferita de traditional , unde avem USER concept.

Common Issues:

- unauthorized access (mine bitcoin for ex,aws account)

Secure by Design

- se adauga la inceput
- Least Privilege Principle
    - users and apps perform only the actions they need to

    - avoid using root accounts
- Reduced Attack Surface
    - minimize the number of public resource
    - remove reduntant users and policies (employees that left)
- Multi-factor Authentication
    - use more than one method to authenticate
- Zero trust Arhitecture
    - add security at each level of the network

Cloud ne lasa sa facem securitate parte din arhitectura

- Network si firewalls - tot resurse
- Users,Roles,Services tot resure
- Fine graine auth controles sunt inforced imediat si sunt in maj caz resure
- WhiteList approach - default user nu poate nimic
- Serviciile behave similar to users
- most requests are http based- evaluare request

User Access and Permissions

- IAM - Identity and Access Management

Users

- multiple users within one accouny
- devs or stakeholders
- some providers offer GROUPS

User PErmissions

- managed through **ROLES** and or **POLICIES**
- might be assigned at group level
- user might be a part of multiple Least Privilege Principle
- ops that require elevated permissions should leverage just in time access (JIT - AWS)

should vary on environment

Application Permissions

- allow a service to access another service
- depind de provider
- azure: serviciul are rol la care pot adauga app
- google cloud: service account

Logging and monitoring

- AWS CloudTrail

Confidentiality

- enforce and democratize encryption (AWS KMS, adica fiecare poate sa cripteze)
- social engineering e tot un risc
- most managed db serv au encryption at rest by default

Integrity

- data integrity checks
- backups
- versioning

Availability

Attack uri: DDos - Distributed Denial of Service

- foloseste servicii dedicate
- protection services/offerings - Azure CDN

Course 10 - Integration Services

Integrare a 2 sau mai multe sisteme

- microservices
- 3rd party APIs
- on-premises systems

Coupling

- dependenta intre entitati (cat de mult entitatile trebuie sa stie despre celelalte pentru a functiona)
- vrem loose coupling Benefits:
- mai usor de mentinut si extins
- mai mare coeziune
- mai eficienta impartirea muncii Challenges:
- necesita tooluri si servicii aditionale
- mai greu de planuit, monitorizat si debugat

Unpredictable Workloads

- scaling up and down

Temporary Issues

- network issues
- billing service down

Syncronicity

- sync com - immediate response
- async com - response later Sync merge cand tot merge. Overcome Async Com issues:
    - Server Push - server trimite notificari

    - Polling - clientul intreaba serverul

Flow Control

- request uriile trebuie sa fie procesate in ordine
- poate temporar oprim requests
- certain requests need to be processed first

Contract definition

- separare interfata de implementare
- easier refactoring
- alte servicii sunt interesate doar de contract
- unified interface

Common Integrations Services

- Queues
- Publish/Subscribe
- Streaming
- API Gateway
- Workflow Services

Queues

Producers send messages to the queue ● Brokers manage the messages ● Consumers poll the queue (pull) ● Any consumer can receive any message(s) (batch) ● Once processed, the consumer sends the acknowledgement (usually deletes the message) ● Messages might be out of order. ● Usually at least once delivery. ● Queues help smoothing out the traffic and avoiding data loss ● Unprocessed message usually go to a deadletter queue

Publish/Subscribe

- Publishers publish messages
- Brokers manage the messages
- Subscribers receive messages (push)
- toti subscriberii primesc mesajul
- some services store mesajele
- services offer message dedup and strict ordering
- dead-letter queue(when message cannot be delivered)

Streaming

- Producers publish messages
- Brokers manage
- Consumers read messages from topics
- fiecare grup de consum primeste o copie a mesajului. partitiile sunt distribuite la conumerii din grup
- toti consumer keep track of messages (checkpoints)
- partitiile sunt vazute ca append only loguri

- mesajele nu sunt sterse dar expir

API Gateway

- usually http based
- entry point door/reverse proxy
- handle requests routing and protocol tran
- add features - rate limiting si caching
- common DP when working with microservices
- usefore for monetizing - API management services

Workflow Services

- orchestrate multiple services
- finite state machine
- usually have lots of conectores/integrations
- span various time intervals
- based on FaaS an leverage other data stores

Course 11 - Application Security

- securitate pentru user Tehnicii comune:
  - auth
  - authorization
  - encryption
  - input validation
  - hashing

Commmon Weaknesses

1. Cross-site scripting (XSS) (46.82)
2. Out-of-bounds write (46.17)
3. Improper input validation (33.47)
4. Out-of-bounds read (26.5)
5. Improper restriction of operations within the bounds of a memory buffer (23.73)
6. SQL injection (20.69)
7. Exposure of sensitive information to an unauthorized actor (19.16)
8. Use after free (18.87)
9. Cross-site request forgery (CSRF) (17.29)
10. OS command injection (16.44)

- maj sunt adresate de framwrokuri noi

Authentication

AuthN si AuthZ

- AuthN - confirmare identitatii

- AuthZ - ce poate face userul - bazat pe roles, permisiuni sau privilegii sunt standardizate

Identity Providers (IdP)

- cand ne inregistram => new identity doar google garanteaza ca suntem owner ul email ului si mai avem si Social Login

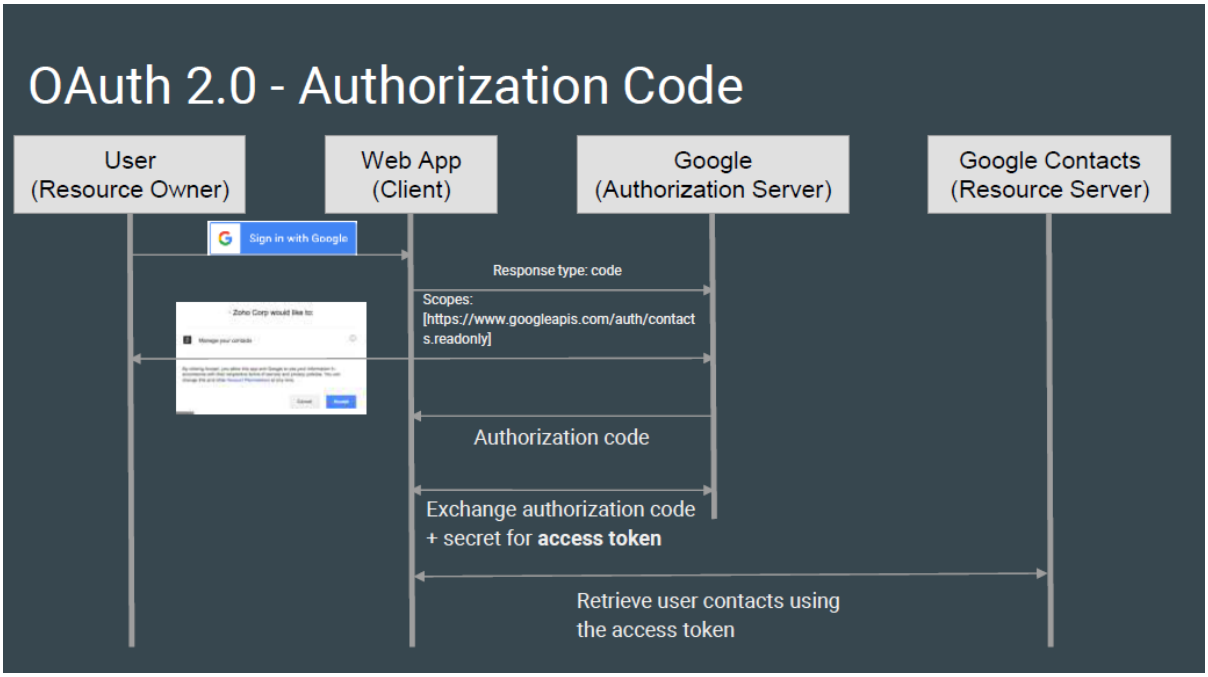
OAuth 2.0

protocol bazat pe widely adopted flows delegare autorizare fara credentaiale implementat de cam toti Resource Owner - User Client - App Resource Server - api pe care vrem sa o accesam Authorization Server - server care da tokenul User Agent - browser

Flows/Grants

- Authorization Code:
    - fol de web apps. requires a server pentru secure info exchange intre client si server
- Implicit:
    - putin secure dar nu ai nevoie de server (obsolete)
    - token e trimis direct de server fara auth code si secret
- Authorization Code with PKCE:
    - similar cu Authorization Code dar mai secure
    - nu are server ca sa faca run la client si sa ii tina secretul
    - se bzeaza pe SPA/mobile app to gen a code challenge si un code berifier
- Resource Owner Password Credentials:
    - userul da direct credentialele
    - special/legacy cases
- Client Credentials:

- pentru secure env



- RPOC si Client Credentials exchange de cred pentru access token

OpenID Connect

- authentication layer peste OAuth 2.0
- merge cu JWT token
- pentru a primi id token adaugam open id la socpes and id token la response type. Server ul de autorizare va trimite un id token care contine info despre user

JWT Tokens

- pot fi validate OFFLINE
- nu trebuie sa faca call la autorization server
- tokens might be out of sync stale
- online valid means calling the auth server each time

e stateless token auth better/easier horizontal scaling pentru very secure scenearios poate le tinem pe server putem sa mai punem si alte info precum roles and perms

Managed Auth Services

Firebase Authentication AWS Cognito Azure Active Directory B2B (standard) and B2C Auth0 Okta OneLogin

SSo and SAML

- SSO - Single Sign On
- SAML - Security Assertion Markup Language
- SAML e un standard pentru schimbarea de autentificare si autorizare intre 2 parti

Encoding, Hashing and Encryption

- Encoding - transformarea datelor in alt format

- Hashing - transformarea datelor in alt format dar nu se poate decripta
- Encryption - transformarea datelor in alt format dar se poate decripta

Encryption

- secure transfer of data(SSL/TLS)
- secure storage of data (data at rest)
- digital signatures 2 types:
    - Symmetric - same key for encrypt and decrypt
    - Asymmetric - public and private key. no large data
- Envelope Encryption - encrypt data with symmetric key and encrypt the symmetric key with asymmetric key

AWS KMS in cloud pot fi handled total de provider sau putem alege our own keys (AWS S3 encryption)

SSL - Secure Sockets Layer

- secure connection between client and server

TLS - Transport Layer Security

- succesor to SSL
- mai secure un server care suporta tls e https enables

Based on RSA

- browser verify autenticiy of server
- secure conection establisted through the ssl/tls handshake

Course 12 - Serverless

- stil servers dar sunt abstracted Consideram serverless daca:
- nu faci setup la infrastructure
- nu ai scale capacity
- pay only what u use
- service can scale to 0

PROS/ CONS

● you don't have to setup any infrastructure ○ Less to manage ✔ ○ Getting started quicker ✔ ○ Less control ✘ ○ Vendor lock-in ✘ ● you don't have to provision/scale capacity ○ Can scale incredibly high ✔ ○ You might want to limit the scale (native ways to limit scale were introduced) ○ Might overload dependencies (e.g. databases) ✘

● you pay only for what you use ○ Scales with your business ✔ ○ You have to pay to keep it running ■ even if it ends up costing more than expected ✘ ● the service can scale down to 0 ○ Great for multiple environments and prototypes ✔ ■ Dev resources are not needed outside of the business hours ○ Cold starts ✘

Classification

- refers to Functions as a Service (FaaS)
- compute
- work flows
- databases
- integrations
- storage
- anaoitice
- monitoring
- dev

Serverless compute

1. FaaS - Function as a Service

- lambda,azure functions,google cloud functions

2. Certain container services

- AWS Fargate, Google Cloud Run

3. Certain PaaS services

- AWS Elastic Beanstalk, Google App Engine

FaaS

- bun pentru event driven apps
- mesaje from integration services
- auth user related actions (send email)
- change in db (useful for nosql db in care data consistency e mantained programay)
- file upload

FaaS for http

- some faas require additional setup(Lambda - API Gateway)
- dar altii au deja url endpoints(Google Cloud Functions)
- cold start - first request

Improve cold starts

- keep the function warm
- call on schedule
- scheduled cu dedicated services sau by using features of Load Balancer sau monitoring

FaaS Limits

resources

- memory -> nu o pot controla

- cpu scales proportionally
- max memory -10 gb lambda,4gb google cloud functions

Deployment size

- couple of MBs

Concurrency

- unele functii for instantiate la nevoie, dar doar pana la un anumit punct
- functiile nu au proces lungi

Databases - Serverless

“Serverless-Native” - designed with the serverless paradigm in mind (NoSQL) ○ No provisioning, just pay for data storage and operations (reads, writes, deletes) ● Hybrid, NoSQL - have a serverless mode alongside a provisioned mode ○ Usually based on some capacity units (e.g. Cosmos DB has request units) that are managed by other services (e.g. AWS Cloudwatch). ● Hybrid, SQL ○ E.g. AWS Aurora Serverless, Azure SQL Database (Serverless Tier) ○ Scaling still takes a bit of time, but services are getting better (e.g. Aurora Serverless v2) Serverless databases tend to be more expensive than their provisioned counterparts (when running continuously)

Course 13 - API Design

API = Application Programming Interface

- can refer to different layers:
  - os functionality
  - com between tiers (orm with db)
  - com between services

API Styles

Procedure / Action oriented

Data sits behind functions ● Similar to writing code in general ● Emphasizes the behavior of the system ● Usually results in large set of procedures (and is ever expanding) ● Each system has its own set of procedures

Resource oriented

Data is the main actor ● The behavior of the system is based on the data ● Similar to working with databases ○ System-specific data model ○ Common well-known features/capabilities ● Usually results in concise set of operations that can be used across multiple use cases

RPC - Remote Procedure call

e implementate de multe protocoale : XML-RPC,JSON-RPC,GRPC

- high performance due low overhead

SOAP - Simble Object Access Protocol

- XML based
- based on RPC WSDL - Web Service Description Language
- defines the contract Pros ● Widely used and adopted ● Establishes an interface contract ○ Important for businesses ● Large number of extensions ○ WS-Addressing, Cons ● XML only ● Not suitable for high performance/throughput requirements ○ XML tends to be verbose and slow to parse ● Tight coupling (since it’s RPC)

REST - Representational State Transfer

- roy fielding
- resources -> urls
- cache and data format through headers